

have `/usr/src/linux`:

```
in -s /usr/src/linux-2.6.27-7 /usr/src/linux
```

- Make the necessary changes to the kernel
- `make clean`
- `make mrproper`
- `make clean`
- Configure the modules. You have to specify the modules to be loaded and those to be compiled in the kernel. It would be ideal to start with your existing configuration file.
- `make dep`
- Modify your Makefile EXTRAVERSION tag to give a unique name to your kernel
- Compile the kernel image

```
nohup make bzImage &
```

```
tail -f nohup.out
```

- Compile the modules

```
nohup make modules 1> modules.out 2> modules.err &
```

```
tail -f modules.err
```

- `make modules_install`
- Copy your kernel image and config file to the `/boot` directory:

```
cp arch/i386/boot/bzImage /boot/cpKernel
```

```
cp .config /boot/cpKernelConfig
```

- Create a new initrd image so that the modules can be loaded while booting. The module created in the `make modules` step will be located in `/lib/modules`:

```
/sbin/mkinitrd /boot/initrd-2.6.27-7cpKernel.img
```

```
/lib/modules/2.6.27-7cpKernel
```

- Now you can edit your `/boot/grub/menu.lst` file (if you are using the GRUB bootloader), so that while booting you can select which kernel to start. Here is a sample entry in `/boot/grub/menu.lst`:

```
timeout 10
```

```
splashimage=/boot/grub/splashimages/firework.xpm.gz
```

```
title VINU, kernel 2.6.27-7
```

```
uuid c21385ba-7c68-4ac0-924f-8bfa6daddc5f
```

```
kernel /boot/vmlinuz-2.6.27-7
```

```
root=UUID=c21385ba-7c68-4ac0-924f-8bfa6daddc5f ro quiet splash
```

```
initrd /boot/initrd.img-2.6.27-7
```

```
quiet
```

```
title VINU, kernel 2.6.27-7
```

```
uuid c21385ba-7c68-4ac0-924f-8bfa6daddc5f
```

```
kernel /boot/vmlinuz-2.6.27-7
```

```
root=UUID=c21385ba-7c68-4ac0-924f-8bfa6daddc5f ro single
```

```
initrd /boot/initrd.img-2.6.27-7
```

```
title VINU, memtest86+
```

```
uuid c21385ba-7c68-4ac0-924f-8bfa6daddc5f
```

```
kernel /boot/memtest86+.bin
```

```
quiet
```

- (Optional step) Delete the entry for the old kernel version in the config file

- Reboot. When the machine restarts, you can select the new kernel!

This is the standard procedure. There are simpler methods as well. (For example, in the case of Debian-based distros, you have a very easy method. Just Google it and find the resources.)

Kernel structure

Within the kernel layer, Linux is composed of five (or six, based on the classification style) major subsystems: the process scheduler (sched), the memory manager (mm), the virtual file system (vfs), the network interface (net), and the inter-process communication (ipc).

Organisation of code

We shall now discuss the way in which the source layout is organised. If you go to the root of the source tree and list the contents, you will see the following directories:

- arch – Contains architecture-specific files
- block – Has the implementation of I/O scheduling algorithms
- crypto – Has the implementation for cipher operations and contains the cryptographic API for implementing encryption algorithms
- Documentation – Contains descriptions about various kernel subsystems
- drivers – Device drivers for various device classes and peripheral controllers can be found here
- fs – Contains the implementation of file systems
- include – Kernel header files
- init – High-level initialisation and start-up code
- ipc – Contains support for Inter-Process Communication (IPC) mechanisms
- kernel – Architecture-independent portions of the base kernel
- lib – Has the library routines
- mm – Holds the memory management implementation
- net – Networking protocols
- scripts – Scripts used for kernel build
- security – Holds the framework for security
- sound – Linux audio subsystem
- usr – Has the initramfs implementation

We will dedicate a few more sessions to reviewing the kernel structure before we meddle with programming. So look out for the forthcoming columns to hack the kernel! 

By: Aasis Vinayak PG

The author is a hacker and a free software activist who does programming in the open source domain. He is the developer of V-language—a programming language that employs AI and ANN. His research work/publications are available at www.aasisvinayak.com